

Introduction to Java

- Java is a **programming language** and a **platform**.

- Java is a high level, object-oriented and secure programming language.

Platform

- Any hardware or software environment in which a program runs, is known as a platform.

- Since Java has a runtime environment (JRE) and API, it is called a platform.

Application (Where Java is used)

- According to Sun, 3 billion devices run Java.
- There are many devices where Java is currently used.

- Some of them are as follows:

Cont...

- Desktop Applications such as acrobat reader, media player, antivirus, etc.
- Web Applications such as irctc.co.in, javatpoint.com, etc.
- Enterprise Applications such as banking applications.

Cont...

- Mobile
- Embedded System

Cont...

- Smart Card
- Robotics

- Games, etc.

Types of Java Applications

- There are mainly 4 types of applications that can be created using Java programming:

-

Cont...

1) **Standalone Application**

Standalone applications are also known as desktop applications or window-based applications.

- These are traditional software that we need to install on every machine.
- **Examples of standalone application** are Media player, antivirus, etc.

Cont...

- AWT and Swing are used in Java for creating standalone applications.

2) Web Application

- An application that runs on the server side and creates a dynamic page is called a web application.

-

Cont...

Currently, Servlet, JSP, Struts, Spring, Hibernate etc. technologies are used for creating web applications in Java.

3) Enterprise Application

- An application that is distributed in nature, such as banking applications, etc. is called enterprise application.
- It has advantages of the high-level security, load balancing etc.

Cont...

- In Java, EJB is used for creating enterprise applications.

4) Mobile Application

- An application which is created for mobile devices is called a mobile application.

•

Cont...

Currently, Android and Java ME are used for creating mobile applications.

Java Platforms / Editions

- There are Following platforms or editions of Java:

1) Java SE (Java Standard Edition)

- It is a Java programming platform.

- It includes Java programming APIs such as java.lang, java.io, java.net, java.util, java.sql, java.math etc.

Cont...

- It includes core topics like **OOPs, String, Regex, Exception, Inner classes, Multithreading, I/O Stream, Networking, AWT, Swing, Reflection, Collection, etc.**

Cont...

2) Java EE (Java Enterprise Edition)

- It is an enterprise platform which is mainly used to develop web and enterprise applications.
- It is built on the top of the Java SE platform.
- It includes topics like Servlet, JSP, Web Services, EJB etc.

Cont...

3) Java ME (Java Micro Edition)

- It is a micro platform which is mainly used to develop mobile applications.

History of Java

- **James Gosling, Mike Sheridan, and Patrick Naughton** initiated the Java language project in June 1991.

- The small team of sun engineers called **Green Team**.
- Initially designed for small, embedded systems in electronic appliances like set-top boxes.

Cont...

- Firstly, it was called "**Greentalk**" by James Gosling, and the file extension was .gt.

- After that, it was called **Oak**.

Cont...

- **Why Oak?** Oak is a symbol of strength and chosen as a national tree of many countries like the U.S.A., France, Germany, Romania, etc.

- In 1995, Oak was renamed as **”Java”**.

Cont...

- **Why java?** Java is an island of Indonesia where the first coffee was produced (called java coffee).

- Java name was chosen by James Gosling while having coffee near his office.

Cont...

- Java was developed by James Gosling, who is known as the father of Java

Features of Java

- The features of Java are also known as java **buzzwords**.

- A list of most important features of Java language is given below.

Cont...

1. Simple
2. Object-Oriented
3. Portable

4. Platform independent

Cont...

5. Secured

6. Robust

7. Architecture neutral

1. Simple

- According to Sun, Java language is a simple programming language because:

- Java syntax is based on C++ (so easier for programmers to learn it after C++).

Cont...

- Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.

- There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

2. Object-oriented

- Java is an object-oriented programming language.

- Java follow the Object-oriented programming (OOPs) concept.

Cont...

- Basic concepts of OOPs are:

A. Class

B. Object

C. Inheritance

Cont...

D. Polymorphism

E. Abstraction

F. Encapsulation

Cont...

A. Class

- A class is a group of objects which have common properties.
- It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

Cont...

- A class in Java can contain:
- **Fields**
- **Methods**
- **Constructors**
- **Blocks**
- **Nested class and interface**

Cont...

B. Object

- An object is an instance of a class.
- An object in Java is the physical because when we are creating a object then memory will be allocated for object
- Object will be created with the help of new keyword.

Cont...

C. Inheritance

- **Inheritance in Java** is a mechanism in which one class inherit the property of another class.

Cont...

D. Polymorphism

- **Polymorphism in Java** is a concept by which we can perform a single action in different ways.
- Polymorphism is derived from 2 Greek words: poly and morphs.

Cont...

- The word "poly" means many and "morphs" means forms.
So polymorphism means many forms.

Cont...

E. Abstraction

- **Abstraction** is a process of hiding the implementation details and showing only functionality to the user.

Cont...

F. Encapsulation

- **Encapsulation in Java** is a process of wrapping code (Function/method) and data together into a single unit,
- for example, a capsule which is mixed of several medicines.



3. Portable

- Java is portable because it facilitates you to carry the Java bytecode to any platform.

4. Platform independent

- Java is platform independent because Java code can be run on multiple platforms,
- for example, Windows, Linux, Sun Solaris, Mac/OS, etc.

Cont...

- Java code is compiled by the compiler and converted into bytecode.
- This bytecode is a platform-independent code because it can be run on multiple platforms, i.e., Write Once and Run Anywhere(WORA).

***Class
File***



***Mac/OS
JVM***

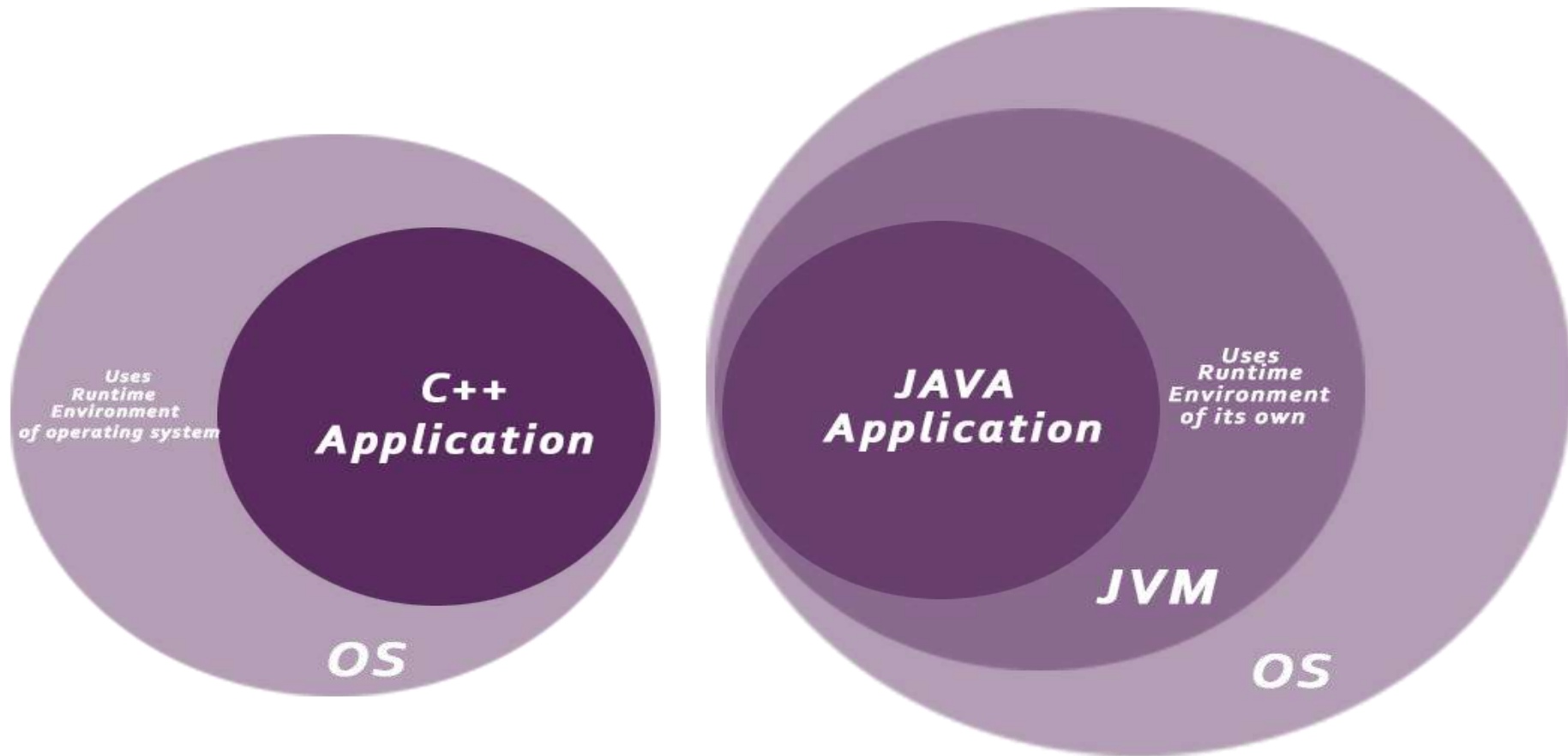
***windows
JVM***

***Linux
JVM***



5. Secured

- Java is secured because:
- **No explicit pointer**
- **Java Programs run inside a virtual machine sandbox**



6. Robust

- Robust simply means strong. Java is robust because:
- It uses strong memory management.
- There is a lack of pointers that avoids security problems.

Cont...

- There is automatic garbage collection in java which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- All these points make Java robust.

7. Architecture-neutral

- Java is architecture neutral because there are no implementation dependent features,

- for example, the size of primitive types is fixed.

Cont...

- In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture.

C++ vs Java

Comparison Index	C++	Java
Platform-independent	C++ is platform-dependent.	Java is platform-independent.
Multiple inheritance	C++ supports multiple inheritance.	Java doesn't support multiple inheritance through class. It can be achieved by interfaces in java.
Call by Value and Call by reference	C++ supports both call by value and call by reference.	Java supports call by value only. There is no call by reference in java.

Structure and Union

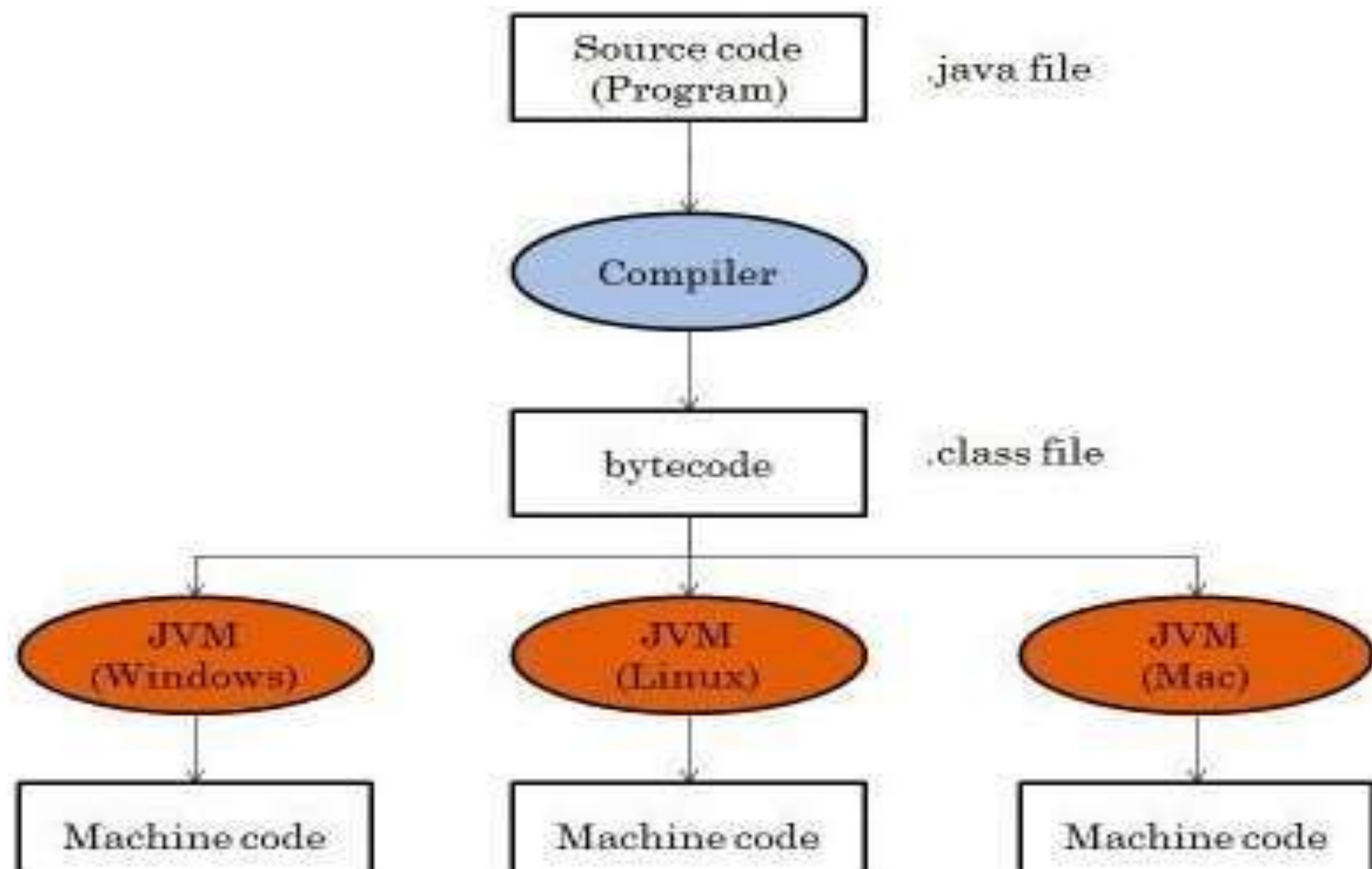
C++ supports structures and unions.

Java doesn't support structures and unions.

Java Bytecode

- Java code is compiled by the compiler and converted into bytecode.

- This bytecode is a platform-independent code because it can be run on multiple platforms, i.e., Write Once and Run Anywhere(WORA).



JVM (Java Virtual Machine)

- JVM (Java Virtual Machine) is an abstract machine.
- It is called a virtual machine because it doesn't physically exist.

- It is a specification that provides a runtime environment in which Java bytecode can be executed.

Cont...

- JVM, JRE, and JDK are platform dependent because the configuration of each OS is different from each other.

- However, Java is platform independent.

Cont...

- The JVM performs the following main tasks:
 - Loads code
 - Verifies code
 - Executes code
 - Provides runtime environment

JVM Architecture (Internal architecture of JVM)

- It contains class loader, memory area, execution engine etc.

Java
Runtime
System



Memory Areas
Allocated by
JVM



Java
Native
Libraries

1) Classloader

- Classloader is a subsystem of JVM which is used to load class files.

- Whenever we run the java program, it is loaded first by the classloader.

2) Class(Method) Area

- Class(Method) Area stores per-class structures such as the runtime constant pool, method data, the code for methods.

Cont...

3) Heap

- It is the runtime data area in which objects are allocated.

4) Stack

- It holds local variables.

Cont...

5) Program Counter Register

- PC (program counter) register contains the address of the Java virtual machine instruction currently being executed.

6) Native Method Stack

- It contains all the native methods used in the application.

Cont...

7) Execution Engine

- It contains:
- **A virtual processor**
- **Interpreter:** Read bytecode stream then execute the instructions.
- **Just-In-Time(JIT) compiler:** It is used to improve the performance.

Cont...

8) Java Native Interface

- Java Native Interface (JNI) is a framework which provides an interface to communicate with another application written in another language like C, C++, Assembly etc.

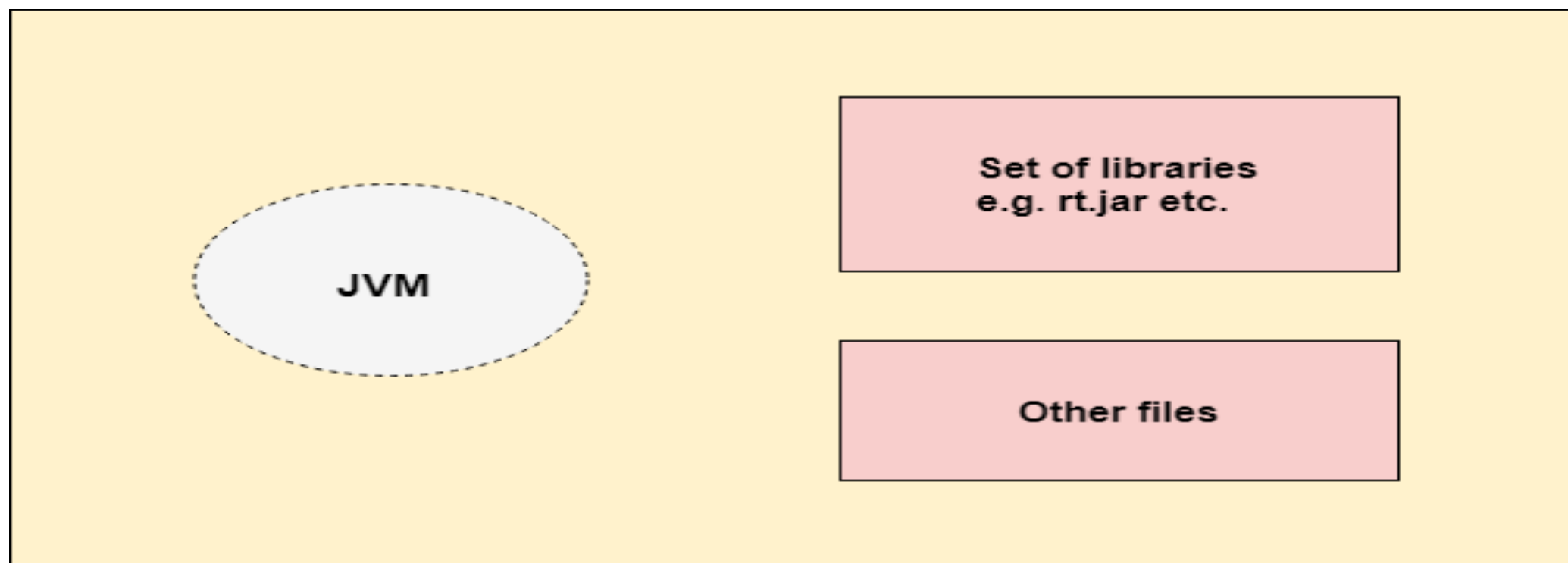
JRE

- JRE Stands for Java Runtime Environment.

- It is used to provide the runtime environment.
- It physically exists.

Cont...

- It contains a set of libraries + other files that JVM uses at runtime.



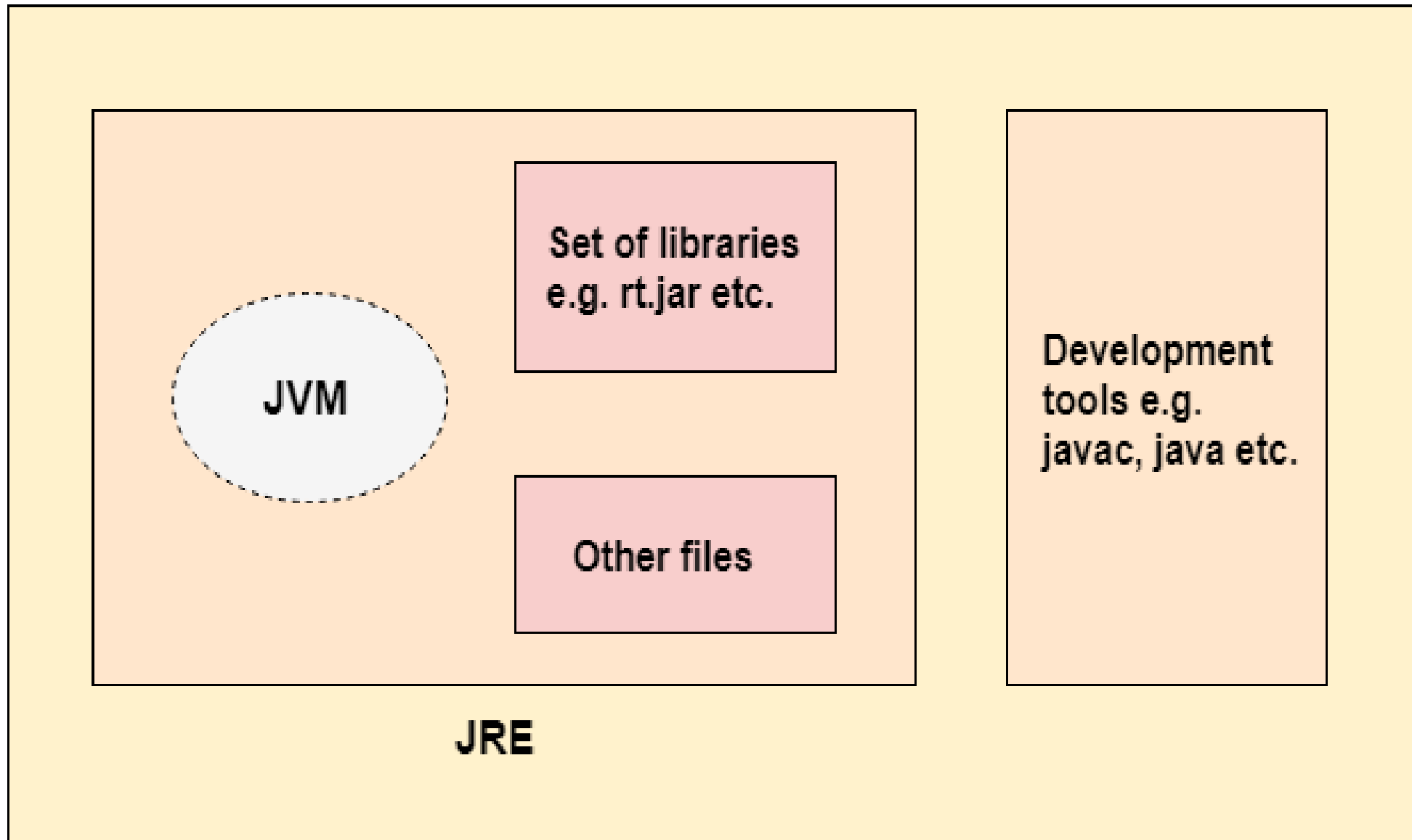
JRE

JDK

- JDK stands for Java Development Kit.
- The Java Development Kit(JDK)is a software development environment which is used to develop Java applications

Cont...

- It physically exists.
- It contains JRE + development tools.



JDK

Just In Time (JIT) Compiler

- The Just-In-Time (JIT) compiler is a an essential part of the JRE i.e. Java Runtime Environment.
- It is responsible for performance optimization of java based applications at run time.

Cont...

- In order to speed up the performance, JIT compiler communicates with JVM at the execution time in order to compile byte code sequences into native machine code.

- Basically, when using JIT Compiler, the native code is easily executed by the hardware.

Cont...

- By doing so, there will be a huge gain in execution speed.

At Compile Time

SourceCode.java → Compiler → ByteCode

At Run Time

Native
Machine
Code

JIT
Compiler



Java Tokens

- A **token** is the smallest element of a program that is meaningful to the compiler.

Types of Tokens

- Keywords
- Identifiers
- Literals
- Operators
- Separators
- Comments

Keywords

- These are the **pre-defined** reserved words of any programming language.
- Each keyword has a special meaning.
- It is always written in lower case. Java provides the following keywords:

01. abstract	02. boolean	03. byte	04. break	05. class
06. case	07. catch	08. char	09. continue	10. default
11. do	12. double	13. else	14. extends	15. final
16. finally	17. float	18. for	19. if	20. implements
21. import	22. instanceof	23. int	24. interface	25. long
26. native	27. new	28. package	29. private	30. protected
31. public	32. return	33. short	34. static	35. super

36. switch	37. synchronized	38. this	39. throw	40. throws
41. transient	42. try	43. void	44. volatile	45. while
46. assert	47. const	48. enum	49. goto	50. strictfp

Cont...

- The keywords **const** and **goto** are reserved but not used.

Identifiers

- Identifiers are used to name a variable, constant, function, class, and array.

- It usually defined by the user.

Cont...

- It uses letters, underscores, or a dollar sign as the first character.

Cont...

- Remember that the identifier name must be different from the reserved keywords.
- There are some rules to declare identifiers are:
- The first letter of an identifier must be a letter, underscore or a dollar sign. It cannot start with digits but may contain digits.
- The whitespace cannot be included in the identifier.

Cont...

- Identifiers are case sensitive.
- Some valid identifiers are:
- PhoneNumber
- PRICE
- radius
- a
- a1
- _phonenumbr

Cont...

- \$circumference
- jagged_array

Cont...

- Invalid identifier names include these:
- 2count
- high-temp
- not/ok

Literals

- In programming literal is a notation that represents a fixed value (constant) in the source code.

- It can be categorized as an integer literal, string literal, Boolean literal, etc.

Cont...

- It is defined by the programmer.

- Once it has been defined cannot be changed.

Cont...

- Java provides five types of literals are as follows:
 - Integer (Ex: 2, 4)
 - Floating Point (Ex: 12.3, 16.8)
 - Character (Ex: 'A', 'a')
 - String (Ex: "Ram", "Hello")

- Boolean (Ex: true, false)

Separators

- The separators in Java is also known as **punctuators**.
- **Square Brackets []**: It is used to define array elements. A pair of square brackets represents the single-dimensional array, two pairs of square brackets represent the twodimensional array.

- **Parentheses ():** It is used to call the functions.

Cont...

- **Curly Braces {}:** The curly braces denote the starting and ending of a code block.
- **Comma (,):** It is used to separate two values and parameters.
- **Assignment Operator (=):** It is used to assign a variable and constant.

Cont...

- **Semicolon (;):** It is the symbol that can be found at end of the statements. It separates the two statements.

- **Period (.):** It separates the package name from the subpackages and class. It also separates a variable or method from a reference variable.

Java Comments

- The Java comments are the statements that are not executed by the compiler and interpreter.

- The comments can be used to provide information or explanation about the variable, method, class or any statement.

Cont...

- It can also be used to hide program code.

Types of Java Comments

- There are three types of comments in Java.
- Single Line Comment
- Multi Line Comment
- Documentation Comment

1) Java Single Line Comment

- The single line comment is used to comment only one line.

- **Syntax:**

`//This is single line comment`

Example:

```
class CommentExample1
{ public static void main(String a[])
    { int i=10; //Here, i is a variable
      System.out.println(i);
    }
}
```

Output: 10

2) Java Multi Line Comment

- The multi line comment is used to comment multiple lines of code.

- **Syntax:**

```
/* This  
   is  
   multi line  
   comment  
*/
```

Example:

```
class CommentExample2
{ public static void main(String a[])
    {
        /* Let's declare and print
           variable in java. */

        int i=10;
        System.out.println(i);
    }
}
```


Output: 10

3) Java Documentation Comment

- The documentation comment is used to create documentation API.
- To create documentation API, you need to use **javadoc tool**.
- **Syntax:**

```
/** This  
    is
```

```
documentation  
comment  
*/
```

Example:

```
public class First  
{  
    /**  
        Main Fuction  
    */ public static void main(String  
        a[])  
    {  
        /** print Function
```

```
*/
```

```
System.out.println("Hello");
```

```
}
```

```
}
```

- Compile it by javac tool:
 - `javac First.java`
- Create DocumentationAPI by javadoc tool:
 - `javadoc First.java`
- Now, there will be HTML files created for your First class in the current directory.

Cont...

Whitespace

- Java is a free-form language.

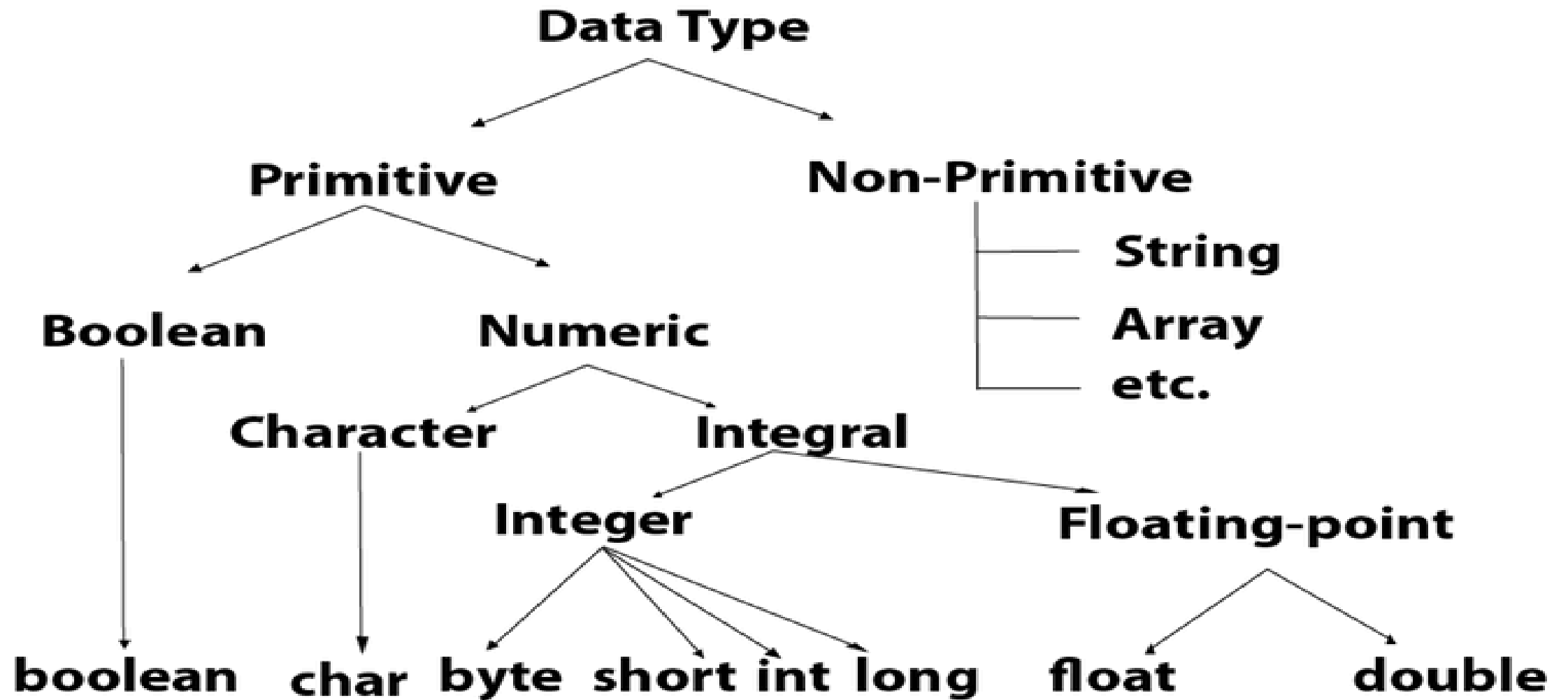
- This means that you do not need to follow any special indentation rules.
- **For example**, Program could have been written all on one line or in any other strange way you felt like typing it, as long as there was at least one whitespace character between each token that was not already delineated by an operator or separator.
- In java, whitespace is a space, tab, or new line.

Cont...

Data Types in Java

- Type of the data is called data type.

- Data types specify the different sizes and values that can be stored in the variable.
- There are two types of data types in Java:
- **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
- **Non-primitive data types:** The non-primitive data types include Classes and Interfaces etc.



Data Type	Default Value	Default size
boolean	false	1 bit
char	'\u0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte

long	0L	8 byte
float	0.0f	4 byte
double	0.0d	8 byte

Boolean Data Type

- The Boolean data type is used to store only two possible values: true and false.

- **Example:** `boolean one = false;`

Why char uses 2 byte in java and what is `\u0000` ?

- It is because java uses Unicode system not ASCII code system.

- The \u0000 is the lowest range of Unicode system

Unicode System

- Unicode is a universal international standard character encoding that is capable of representing most of the world's written languages.

- In unicode, character holds 2 byte, so java also uses 2 byte for characters.
- lowest value:\u0000
- highest value:\uFFFF

Java Variables

- **Variable** is name of reserved area allocated in memory.
- In other words, it is a name of memory location.

- A variable is a container which holds the value.

Cont...

- A variable is assigned with a data type.

- It is a combination of "vary + able" that means its value can be changed.

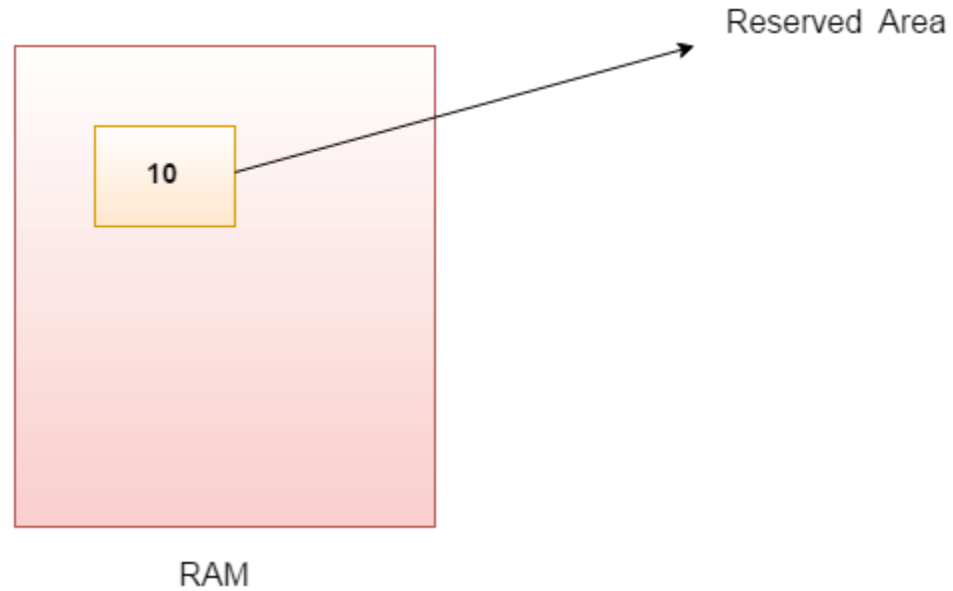
Declaration of a Variable

- **Syntax:** datatype variablename ;

- **Example:** `int data;`

Initialization of a Variable

- **Syntax:** `datatype variablename = value ;`
- **Example:** `int data = 10;`

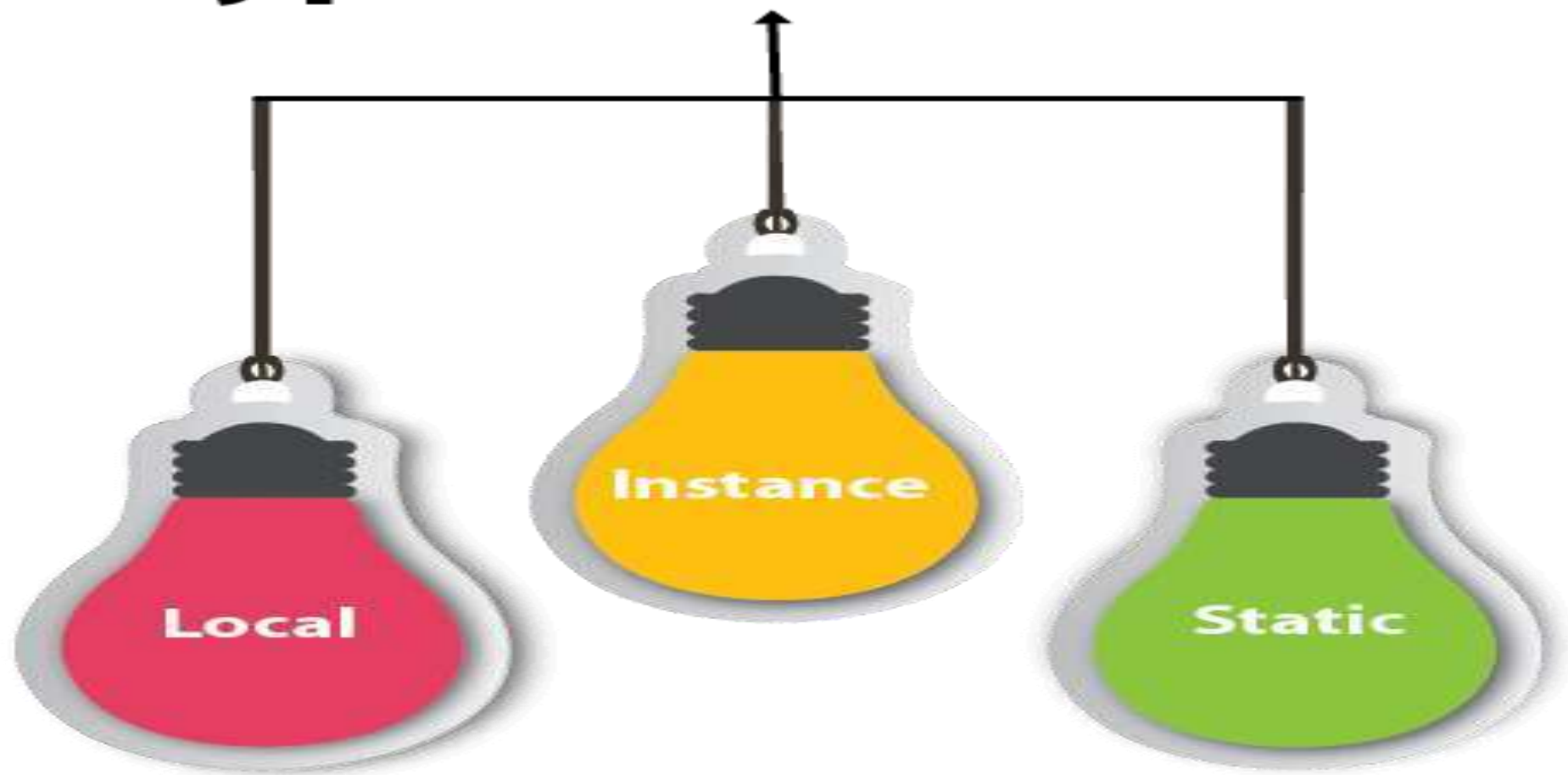


Types of Variables

- There are three types of variables in Java:
- local variable

- instance variable
- static variable

Types of Variables



1) Local Variable

- A variable declared inside the body of the method, block and constructor is called local variable.

- You can use local variable only within that method, block and constructor in which it is declared.

Cont...

- A local variable cannot be defined with "static" keyword.

2) Instance Variable

- A variable declared inside the class but outside the body of the method, block and constructor is called instance variable.

- It is not declared as static.

3) Static variable

- A variable which is declared as static is called static variable.

- It can not be local.

Cont...

- You can create a single copy of static variable and share among all the instances (object) of the class.

- Memory allocation for static variable happens only once when the class is loaded in the memory.

Example

```
class A
{
    int data=50;//instance variable
    static int m=100;//static variable
    void method()
    {
```

```
        int n=90;//local variable  
    }  
} //end of class
```

Java Constant

- **Constant** is a value that can not be changed after assigning it.

- In Java, to declare any variable as constant, we use final modifiers (keyword).

Syntax to declare a constant :

- **final** datatype variablename = value;

- **Example: final double PRICE=432.78;**

Example

```
class HelloWorld
{
    public static void main(String a[])
    {
        final int b =60;
        System.out.println(b);
    }
}
```

Type Casting in Java

- Convert a value from one data type to another data type is known as **type casting**.

Types of Type Casting

- There are two types of type casting:
- Widening Type Casting
- Narrowing Type Casting

Widening Type Casting

- Converting a lower data type into a higher one is called **widening** type casting.
- It is also known as **implicit conversion** or **upcasting**.

- It is done automatically.

Cont...

- It takes place when:
- Both data types must be compatible with each other.
- The target type must be larger than the source type.

• **byte -> short -> char -> int -> long -> float -> double**

Example

```
class A
{ public static void main(String a[])
    { int x = 7;
        //automatically converts the integer type into long type
        long y = x;
        //automatically converts the long type into float type
        float z = y;
        System.out.println(x);
        System.out.println(y);
        System.out.println(z);
    }
}
```

```
}  
  }
```

Output

7

7

7.0

Narrowing Type Casting

- Converting a higher data type into a lower one is called **narrowing** type casting.
- It is also known as **explicit conversion** or down **casting**.

- It is done manually by the programmer.

Cont...

- If we do not perform narrowing type casting then the compiler reports a compile-time error.

• **double -> float -> long -> int -> char -> short -> byte**

Example

```
class A
{ public static void main(String a[])
    { double d = 166.66;
        //converting double data type into long data type
        long l = (long)d;
        //converting long data type into int data type
        int i = (int)l;
        System.out.println(d);
        //fractional part lost
        System.out.println(l);
        //fractional part lost
        System.out.println(i);
```

}

}

Output

166.66

166

166

Java Command Line Arguments

- The java command-line argument is an argument i.e. passed at the time of running the java program.

- The arguments passed from the console can be received in the java program and it can be used as an input.

Simple example of command-line argument in java

```
class A  
{ public static void main(String a[])  
    {
```

```
System.out.println(a[0]);
```

```
}
```

```
}
```

Compile

javac

A.java

run java

A 10

Output

10

Java Arrays

- Normally, an array is a collection of similar type of elements which has contiguous memory location.

- **Java array** is an object which contains elements of a similar data type.

Cont...

- Additionally, The elements of an array are stored in a contiguous memory location.

- Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.

Cont...

- Unlike C/C++, we can get the length of the array using the **length member**.

Types of Array in java

- There are two types of array.
- Single Dimensional Array
- Multidimensional Array

Single Dimensional (1-D) Array in Java

- An Array that will contain only one subscript is called Single Dimensional Array.

Syntax to Declare 1-D Array in Java

- `dataType arrayname[]; (or)`
- `dataType[] arrayname; (or)`
- `dataType []arrayname;`

Example

- `int a[];`

Instantiation of 1-D Array in Java

- **Syntax:**
- `arrayname = new datatype[size];`

- **Example:**
- `a = new int[2];`

Declaration and instantiation in one line

- **Example:**
- `int a[] = new int[2];`

Example of Java 1-D Array

```
class Testarray
{ public static void main(String a1[])
    { int i;
        int a[]=new int[2]; //declaration and instantiation
        a[0]=10;           //initialization
        a[1]=20;

        //traversing array for(i=0; i<=(a.length)-1; i++) //length is the
        property of array System.out.println(a[i]);
    }
}
```

Declaration, Instantiation and Initialization of Java 1-D Array

- We can declare, instantiate and initialize the java array together.

- **Example:**
- `int a[]={33,3};`//declaration, instantiation and initialization

For-each Loop for Java Array

- The Java for-each loop prints the array elements one by one.

The syntax of the for-each loop is given below:

```
for(datatype variablename:arrayname)
{
    //body of the loop
}
```

Example

```
//Java Program to print the array elements using for-each loop class
Testarray1
```

```
{ public static void main(String a[])  
    { int arr[]={33,3};  
        //printing array using for-each loop  
        for(int i : arr)  
            System.out.println(i);  
    }  
}
```

Multidimensional Array in Java

- An Array that will contain more than one subscript is called Multi Dimensional Array.

Types of Multidimensional Array

- 2-D Array
- 3-D Array
- n-D Array

2-D Array

- An Array that will contain two subscript is called Two Dimensional Array.

Syntax to Declare 2-D Array in Java

- `dataType[][] arrayname; (or)`
- `dataType [][]arrayname; (or)`
- `dataType arrayname[][];`
- **Example**
- `int a[][] ;`

Instantiation of 2-D Array in Java

- **Syntax:**

- `arrayname = new datatype[rowsize][columnsize];`

- **Example:**

- `a = new int[2][2];`

-

Declaration and instantiation in one line

- **Example:**

- `int a[][] = new int[2][2];`

Example to initialize 2-D Array in Java

- `arr[0][0]=1;`
- `arr[0][1]=2;`
- `arr[1][0]=3;`
- `arr[1][1]=4;`

Declaration, Instantiation and Initialization of Java 2-D Array

- We can declare, instantiate and initialize the java array together.
- **Example:**
- `int a[][]={{1,2},{3,4}};`//declaration, instantiation and initialization

Java String

- Generally, String is a sequence of characters.
- But in Java, string is an object that represents a sequence of characters.
- The String class is used to create a string object.

How to create a string object?

- There are two ways to create String object:
- By string literal
- By new keyword

1) String Literal

- Java string literal is created by using double quotes.

- Example:
- `String s="welcome";`

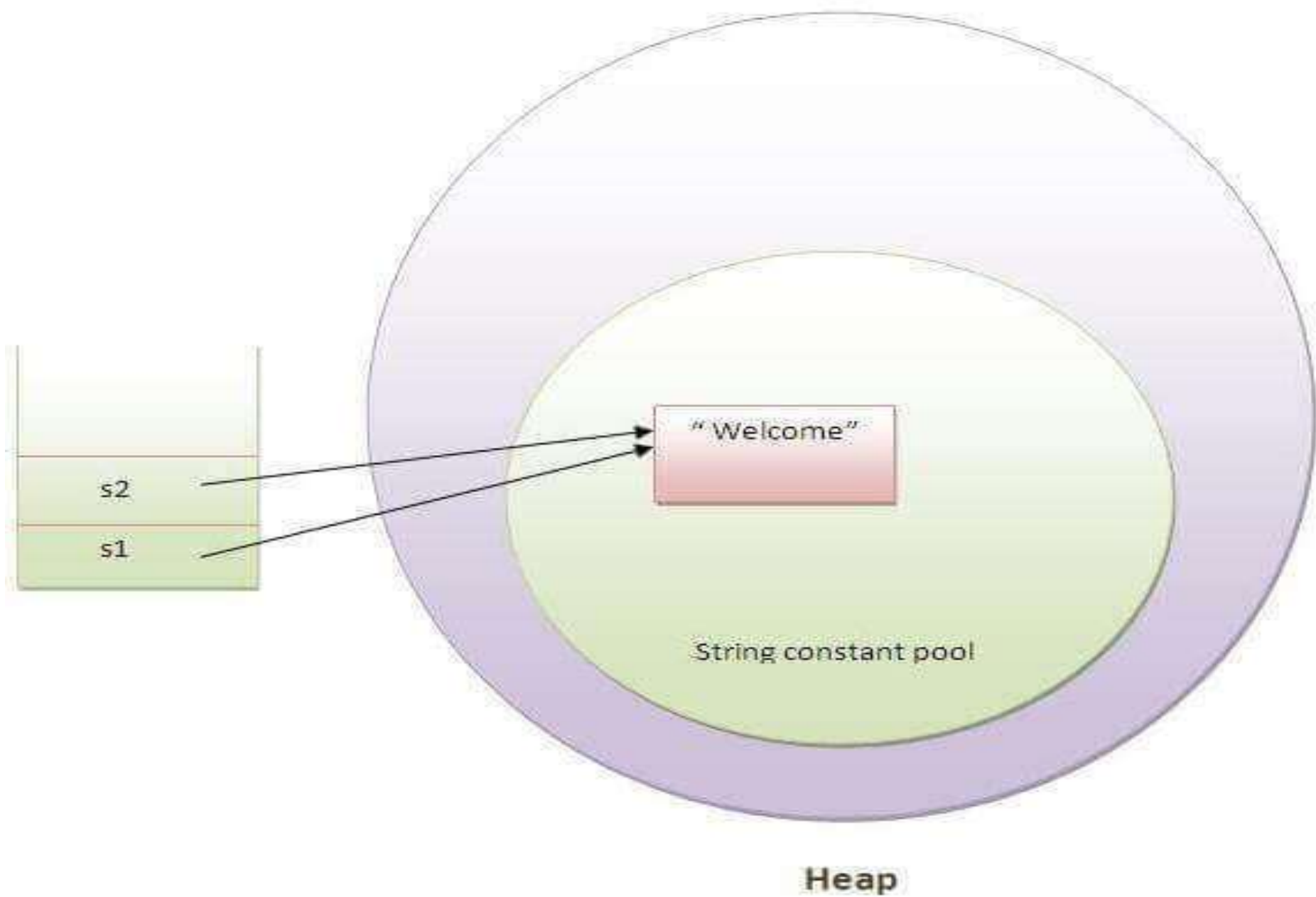
Cont...

- Each time you create a string literal, the JVM checks the "string constant pool" first.
- If the string already exists in the pool, a reference to the pooled instance is returned.

- If the string doesn't exist in the pool, a new string instance is created and placed in the pool.

Cont...

- Example:
- `String s1="Welcome";`
- `String s2="Welcome";`//It doesn't create a new instance



Why Java uses the concept of String literal?

- To make Java more memory efficient (because no new objects are created if it exists already in the string constant pool).

2) By new keyword

- Example:

```
String s=new String("Welcome");
```

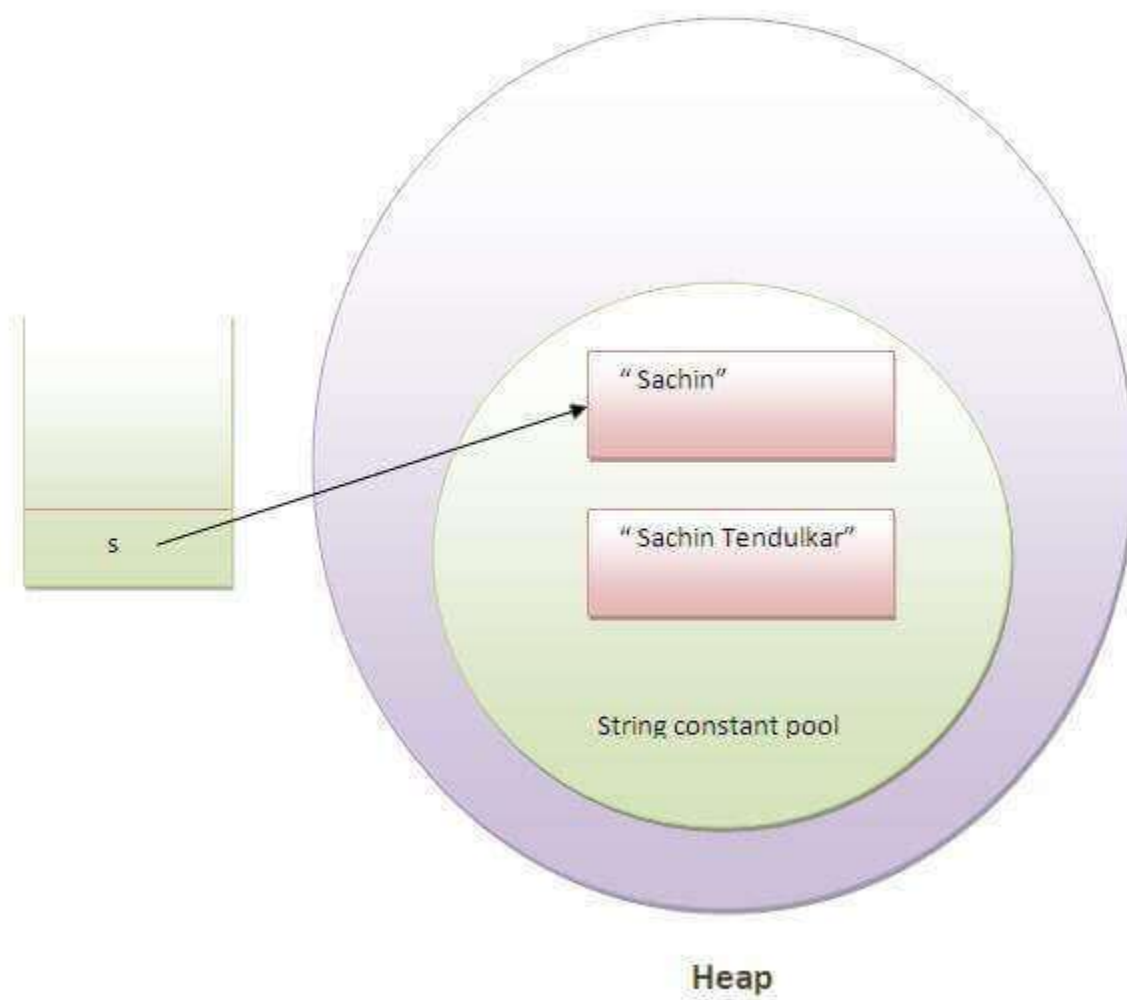
Immutable String in Java

- In java, **string objects are immutable**. Immutable simply means unmodifiable or unchangeable.
- Once string object is created its data can't be changed but a new string object is created.

Example

```
class A
{ public static void main(String args[])
    {
        String s="Sachin";
        s.concat(" Tendulkar");//concat() method appends the string at the end
        System.out.println(s);//will print Sachin because strings are immutable objects
    }
}
```

Output:Sachin



Why string objects are immutable in java?

- Because java uses the concept of string literal.
- Suppose there are 2 reference variables, all refers to one object "sachin".

- If one reference variable changes the value of the object, it will be affected to all the reference variables. That is why string objects are immutable in java.

Java String class methods

- The String class provides many useful methods to perform operations on sequence of char values (String).

Cont...

1. `char charAt(int index)`: returns char value for the particular index.

Example

```
class CharAtExample
{ public static void main(String args[])
    {
        String name="javatpoint"; char ch=name.charAt(4);//returns the
        char value at the 4th index System.out.println(ch);
    }
}
```

Output: t

Cont...

2. `int length()`: returns string length.

Example:

```
class LengthExample
{ public static void main(String a[])
    {
        String
        s1="javatpoint"; int x ; x
        = s1.length();
        System.out.println(x); //10 is the length of javatpoint string
    } } Output:
10
```

Cont...

3. `String substring(int beginIndex, int endIndex)`: returns substring for given begin index and end index.

```
class SubstringExample
{ public static void main(String a[])
{
    String s1="javatpoint", s2;
    s2 = s1.substring(2,4);
    System.out.println(s2);//returns va
}
}
```

Output:

va

Example

Cont...

- `String toLowerCase()`: returns a string in lowercase.

```
class StringLowerExample
{ public static void main(String a[]) {
    String s1="JAVATPOINT HELLO string", s2; s2
    = s1.toLowerCase();
    System.out.println(s2);
}
}
```

Output:

javatpoint hello string

Example

Cont...

- `String toUpperCase()`: returns a string in uppercase.

```
class StringUpperExample
{ public static void main(String a[]) {
    String s1="hello string", s2;
    s2    =    s1.toUpperCase();
    System.out.println(s2);
}
}
```

Output:

HELLO STRING

Example

String Concatenation in Java

- There are two ways to concat string in java:
- By + (string concatenation) operator
- By concat() method

1) String Concatenation by + (string concatenation) operator

- Java string concatenation operator (+) is used to add strings.

Example 1

```
class TestStringConcatenation1
{ public static void main(String a[])
    {
        String s="Sachin"+"Tendulkar";
        System.out.println(s);
    }
}
```

Output: SachinTendulkar

Example 2

```
class TestStringConcatenation2
{ public static void main(String a[])
    {
        String s=50+30+"Sachin"+40+40;
        System.out.println(s);
    }
}
```

Output: 80Sachin4040

Cont...

- **Note:** After a string literal, all the + will be treated as string concatenation operator.

2) String Concatenation by concat() method

- The String concat() method concatenates the specified string to the end of current string.
- **Syntax:**
- **public** String concat(String another)

Example

```
class TestStringConcatenation3
{ public static void main(String a[])
    {
        String s1="Sachin";
        String s2="Tendulkar";
        String s3=s1.concat(s2);
        System.out.println(s3);
    }
}
```

Output: SachinTendulkar

Class and Object

Class

- Class is a logical entity because when we are creating a class then memory will not be created for a class.
- Class is a collection of data members, constructors, methods and blocks.

Cont...

A class is a group of objects which have common properties.

Objects

- Object is a physical entity because when we are creating object then memory will be allocated for object.

- Object is created with the help of new keyword.

Cont...

Object is the instance of a class.



Constructors in Java

- In Java, a constructor is a block of codes similar to the method.
- It is called when object is created.
-

- At the time of calling constructor, memory for the object is allocated in the memory.

Cont...

It will be used to initialize the object.



Rules for creating Java constructor

- Constructor name must be the same as its class name.

•

- A Java constructor cannot be abstract, static, final, and synchronized

Types of Java Constructors

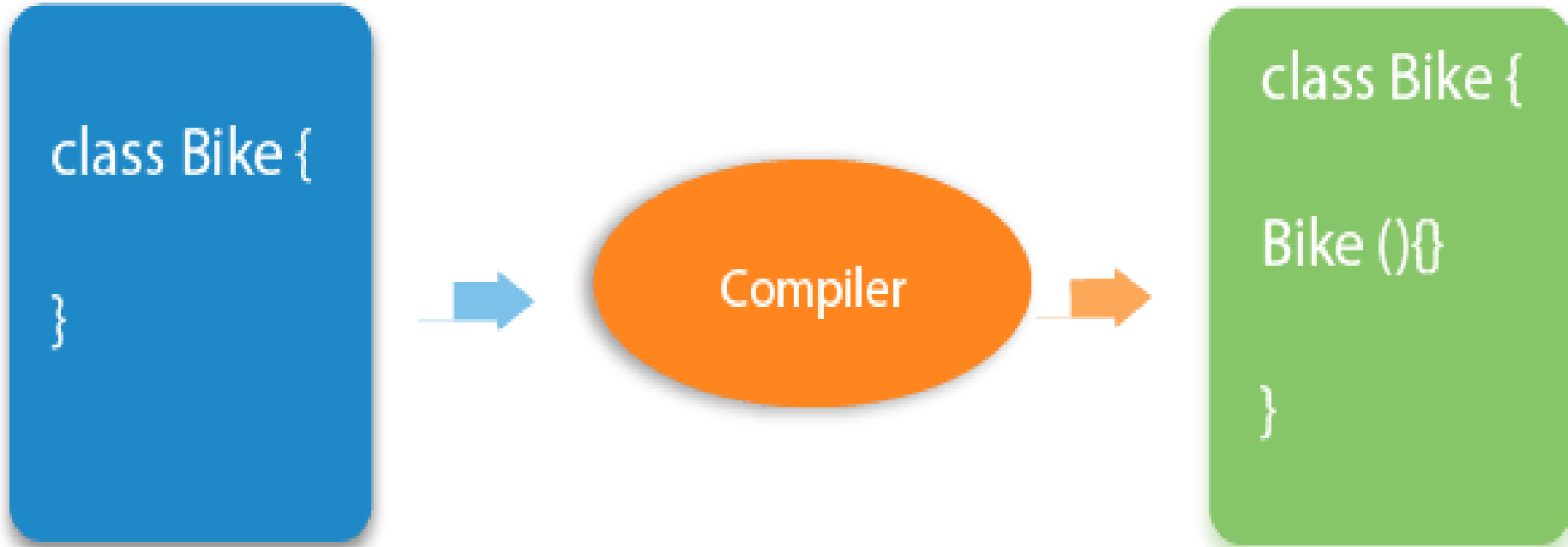
- There are 3 Types of constructors.
- **Default Constructor.**
- **No-argument Constructor.**
- **Parameterized Constructor.**

1. Default Constructor.

- If there is no constructor in a class, compiler automatically creates a default constructor.

Cont...





Example

```
class Student
{ int id;  public static void main(String
    a[])
    {
        Student s1=new Student();
        System.out.println(s1.id);
    }
}
```

Output: 0

2. No-argument Constructor.

- A constructor that has no parameter is known as no-argument constructor.

Example

```
class Bike
{
    Bike()
    {
        System.out.println("Bike is created");
    }
    public static void main(String a[])
    {
```

```
        Bike b=new Bike();  
    }  
}
```

Output: Bike is created

3. Parameterized Constructor

- A constructor which has a specific number of parameters is called a parameterized constructor.



Example

```
class Student
{
    int id;
    String name;
    Student(int i, String n)
    {
        id = i;
        name = n;
    }
    void display()
    {
        System.out.println(id);
        System.out.println(name);
    }
    public static void main(String a[])
    {
        Student s1 = new Student(111,"Karan");
```



```
}  
    s1.display();  
}
```

Cont...

Output:

111

Karan



Difference between constructor and method in Java

Constructor	Method
A Constructor is called implicitly by the system.	A Method is called by the programmer.
A Constructor is called when a object is created using the keyword new .	A Method is called through method calls.

A Constructor's name must be same as the name of the class.

A Method's name can be anything.

A Constructor doesn't have a return type.

A Method must have a return type.

Ways to initialize object:

There are 3 ways to initialize object in Java.

- By reference variable

- By method
- By constructor

1) Initializing an Object by reference variable

- Initializing an object means storing data into the object.

Example

```
class Student
{
    int id;
    public static void main(String a[])
    {
        Student s1= new Student();
        s1.id=101;
        System.out.println(s1.id);
    }
}
```

Output

101

2) Initializing an Object by method

Example

```
class Student
{
    int id;
    void display(int n)
    {
        id = n;
        System.out.println(id);
    } public static void main(String
    a[])
    {
        Student s1=new Student();
```

```
    s1.display(101);  
}  
}
```

Cont...

Output:

101



3) Initializing an Object by a constructor

Example



```
class Student
{ int id;
  Student(int
    i)
  { id = i;
  }
  void display()
  {
    System.out.println(id);
  }
  public static void main(String a[])
  {
```

```
}  
}  
  
Student s1 = new Student(101);  
s1.display();  
}
```

Cont...

Output:

101



Java static keyword

- The **static keyword** in Java is used for memory management mainly.



- We can apply static keyword with variables, methods and blocks etc.



Cont...

- The static can be:
- Variable (also known as a class variable)
- Method (also known as a class method)
- Block

1) Java static variable

- If you declare any variable as static, it is known as a static variable.
- The static variable can be used to refer to the common property of all objects (which is not unique for each object).

- for example, college name of students, etc.

Cont...

- The static variable gets memory only once in the class area at the time of class loading.



Advantage of static variable

- It makes your program **memory efficient** (i.e., it saves memory).

Example: Understanding the problem without static variable

```
class Student
{
    String collegename;
    Student(String c)
    {
        collegename = c;
    }
    void display ()
    {
        System.out.println(collegename);
    }
    public static void main(String
a[])
    {
```

```
Student s1 = new Student("GLAU"); Student  
s2 = new Student("GLAU");  
s1.display(); s2.display();  
} }
```

Cont...

Output:

GLAU

GLAU



Example of static variable

```
class Student
{
    static String collegename = "GLAU"; //static variable
    void display ()
    {
        System.out.println(collegename);
    }
    public static void main(String
a[])
    {
        Student s1 = new Student();
        Student s2 = new Student();
        s1.display(); s2.display();
    }
}
```


Cont...

Output:

GLAU

GLAU



2) Java static method

- If you apply static keyword with any method, it is known as static method.

- A static method can be called without the need for creating an instance (object) of a class.



Restrictions for the static method

- There are two main restrictions for the static method. They are:
- The static method can not use non static data member or call non-static method directly.
- this and super cannot be used in static context.

Example of static method

```
class A
{ static void display()
    {
        System.out.println("Static Method");
    }
}
class B
{ public static void main(String a[])
    {
        A.display();
    }
}
```

```
}  
    }  
}
```

Cont...

Output:

Static Method



3) Java static block

- It is used to initialize the static data member.

- It is executed before the main method at the time of class loading.



Example of static block

```
class A
{
    static
    {
        System.out.println("static block");
    }
    public static void main(String
a[])
    {
```

```
}  
  
        System.out.println("Hello main"); }  
}
```

Cont...

Output:

static block

Hello main



this keyword in java

- this is a **reference variable** that refers to the current class object.

Usage of java this keyword

- There are many usage of java this keyword.
- this can be used to refer current class instance variable.

- this() can be used to call current class constructor.

1) this: to refer current class instance variable



- The **this keyword** can be used to refer current class instance variable.

- If there is ambiguity between the instance variables and parameters (local variable), this keyword resolves the problem of ambiguity.

Example: Problem without this keyword

```
class Student
{ int x; int y;
    Student(int x, int y)
    { x = x; y = y;
    }
    void display()
    {
        System.out.println(x);
        System.out.println(y);
    }
}
```

```
}  
  
public static void main(String a[])  
{  
    Student s1=new Student(10,20);  
    s1.display();  
}  
}
```

Cont...

Output: 0
0



Ex: Solution of Previous Problem with this keyword

```
class Student
{ int x; int y;
    Student(int x, int y)
    { this.x = x; this.y
        = y;
    }
    void display()
    {
        System.out.println(x);
        System.out.println(y);
    }

    public static void main(String a[])
    {
```

```
Student s1=new Student(10,20);  
s1.display();
```

```
}
```

```
}
```

Cont...

Output: 10
20



2) this() : to call current class constructor

- The this() can be used to call the current class constructor.
- Call to this() must be the first statement in constructor.

Example 1

```
class A
{
    A()
    {
        this(50);
        System.out.println("hello a");
    }
    A(int x)
    {
        System.out.println(x);
    }
    public static void main(String a1[])
    {
    }
```

```
{  
    A a=new A();  
}  
}
```

Cont...

Output:

50

hello a



Inheritance in Java

- Inheritance is a process to inherit the property of one class (super class or parent class) into another class (sub class or child class) is called Inheritance.

Cont...

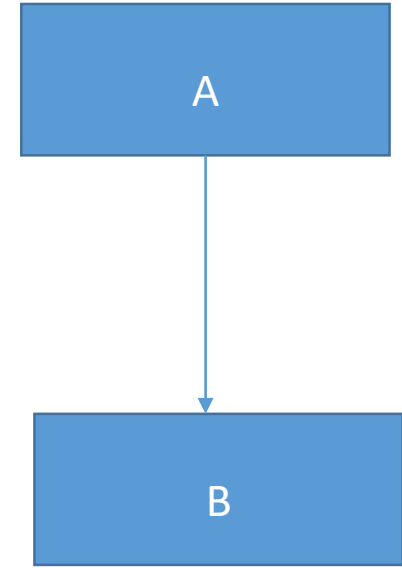
- extends keyword will be used to inherit the property of one class into another class.

- **The syntax of Java Inheritance**

class B extends A

{

 //methods and fields }



Types of inheritance in java

- On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.

- In java programming, multiple and hybrid inheritance is supported through interface only.
- Multiple inheritance is not supported in Java through class.

1. Single Inheritance

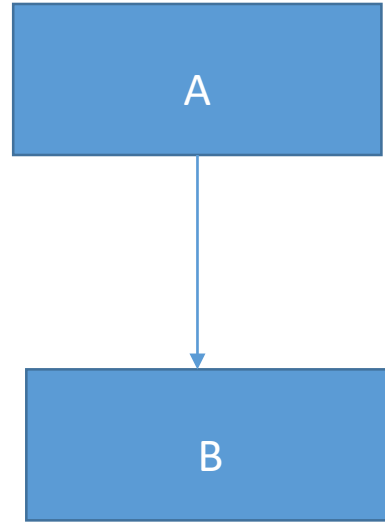
- When a class inherits another class, it is known as a single inheritance.



Example

```
class A
{ void display()
  {
  } }
class B extends
A
```

```
System.out.println("Class A");
```



```
{ void printf()
    {
        System.out.println("Class B");
    }
    public static void main(String a[])
    {
        B b1 = new B();
        b1.display();
        b1.printf();
    }
}
```


Cont...

Output: Class

A

Class B



Multilevel Inheritance

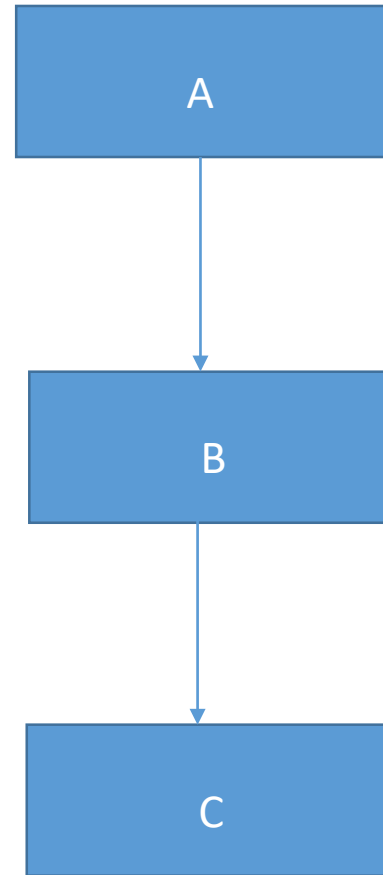
- When there is a chain of inheritance, it is known as multilevel inheritance.

- As you can see in the example given next slide, A class inherits by B class which again inherits by C class, so there is a multilevel inheritance.

Cont...



Example
class A



```
{ void display()
    {
        System.out.println("Class A");
    }
}
```

```
class B extends A
{ void printf()
    {
        System.out.println("Class B");
    }
}
```

Cont...

```
class C extends B
{
    void show()
    {
        System.out.println("Class C");
    }
    public static void main(String a[])
    {
```

```
    C c1 = new C();  
    c1.display();  
    c1.printf(); c1.  
    show();  
}  
}
```

Cont...

Output:

Class A

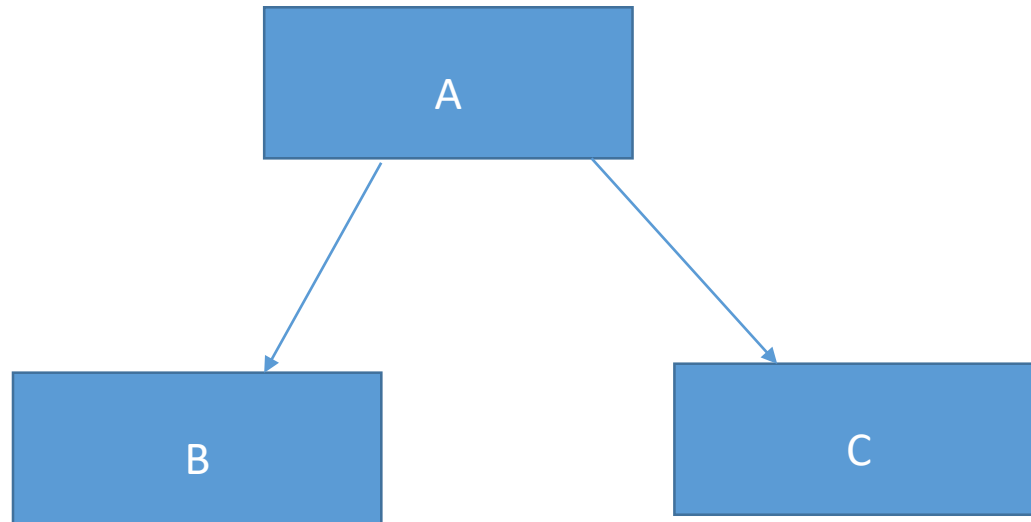
Class B



Class C

Hierarchical Inheritance

- When two or more classes inherit a single class, it is known as hierarchical inheritance.



Example

```
class A
{ void display()
    {
        System.out.println("Class A");
    }
}
```

```
class B extends A
{ void printf()
    {
        System.out.println("Class B");
    }
}
```

```
}  
}
```

Cont...

```
class C extends A  
{  
    void show()  
    {  
        System.out.println("Class C");  
    }  
    public static void main(String a[])
```



```
{  
    C c1 = new C();  
    c1.display();  
    c1.show();  
}  
}
```

Cont...

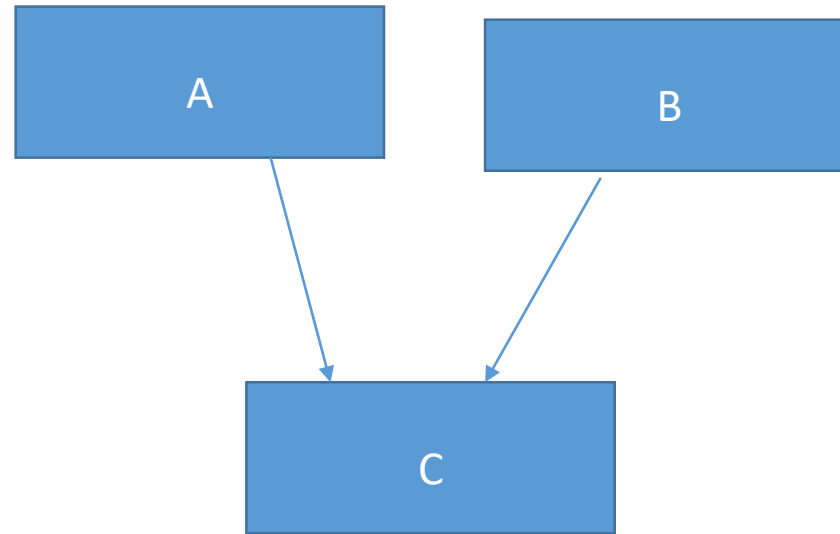
Output:
Class A
Class C



Multiple Inheritance

- When one class inherits multiple classes, it is known as multiple inheritance.
- Multiple inheritance is not supported in Java through class.

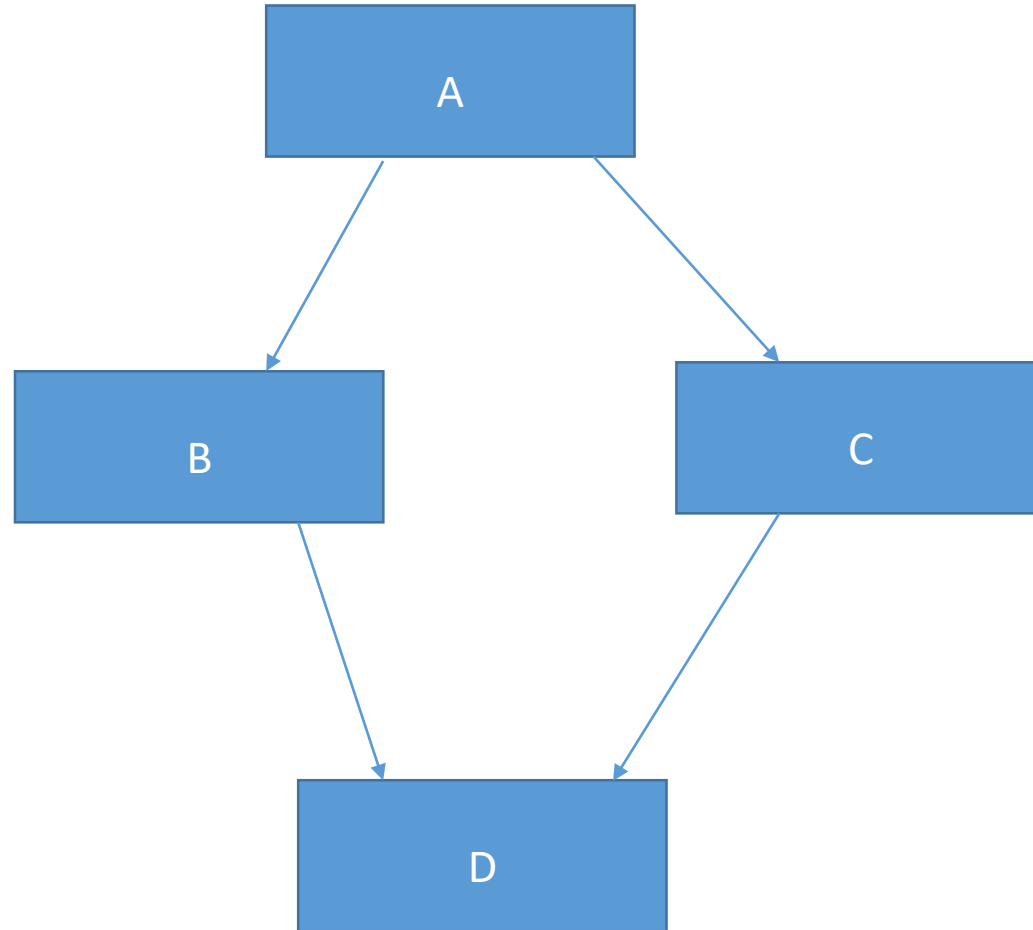
Cont...



Hybrid inheritance

- A hybrid inheritance is a combination of more than one **types of inheritance**.
- Hybrid inheritance is not supported in Java through class.

Cont...



Why use inheritance in java

- For Code Reusability.



- For Method Overriding.

Method Overloading and Overriding

Method Overloading in Java

- If a class has multiple methods having same name but different in parameters, it is known as **Method Overloading**.

- In Java, Method Overloading is not possible by changing the return type of the method only.

Different ways to overload the method

- There are two ways to overload the method in java
- By changing number of arguments
- By changing the data type

Example

```
class Adder  
{ void add(int a, int b)
```

```
{ int c = a+b;
    System.out.println(c)
    ;
}
void add(int a, int b, int c)
{ int d = a+b+c;
    System.out.println(d)
    ;
}
public static void main(String x[])
{
    Adder a1 = new Adder();
    a1.add(10,20);
    a1.add(10,20,30);
}
}
```

Cont...

Output:

30

60

Advantage of method overloading

- Method overloading increases the readability of the program.

Method Overriding in Java

- If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in Java**.

Rules for Java Method Overriding

- The method must have the same name as in the parent class.
- The method must have the same parameter as in the parent class.
- There must be an IS-A relationship (inheritance).

Example of method overriding

```
class Vehicle
{
    void run()
    {
        System.out.println("Vehicle is running");
    }
}
class Bike extends Vehicle
{ void run()
    {
        System.out.println("Bike is running safely");
    } public static void main(String
x[])
    {
```

```
        Bike obj = new Bike();  
        obj.run();  
    } }
```

Cont...

Output:

Bike is running safely

Usage of Java Method Overriding

- Method overriding is used for runtime polymorphism.

Dynamic Method Dispatch

- When the object of child class is initialized to the reference variable of parent class is known as dynamic method dispatch.
- It is also called Upcasting.

Example

```
class A
{
    void display()
    {
        System.out.println("A");
    }
    void paint()
    {
        System.out.println("paint");
    }
}
```

Cont...

```
class B extends A
{ void display()
    {
        System.out.println("B");
    } void
print()
    {
        System.out.println("print");
    }
public static void main(String x[])
{ A a1 = new
  B();
  a1.display();
  a1.paint();
```

```
}  
}  
Output:  
B  
paint
```

Abstraction in Java

Abstraction in Java

- Abstraction is a process of hiding the implementation details and showing only functionality to the user.

- For example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.

Ways to achieve Abstraction

- There are two ways to achieve abstraction in java.
 1. Abstract class (0 to 100%)
 2. Interface (100%)

1. Abstract class in Java

- A class which is declared as keyword abstract is known as an **abstract class**.

- It can have abstract and non-abstract methods.

Points to Remember

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.

- We can not create the object of abstract class.

Cont...

- It needs to be extended in another class and its abstract method should be implemented in that class.

Example of abstract class

```
abstract class A  
{  
  
}
```

Abstract Method in Java

- A method which is declared as abstract and does not have body is known as an abstract method.

- **Example of abstract method abstract void A();//no method body and abstract**

Example

```
abstract class A
{
    abstract void run();
    void display()
    {
        System.out.println("Bike");
    }
}
class B extends A
{
    void run()
    {
```

```
        System.out.println("running safely");
    }
    public static void main(String args[])
    {
        B b1 = new B(); b1.run();
        b1.display();
    }
}
```

Cont...

Output:

running safely

Bike

2. Interface in Java

- It has only abstract methods, not method with body.
- It is used to achieve 100 % abstraction and multiple inheritance in Java.

Why use Java interface?

- It is used to achieve 100% abstraction.
- By interface, we can support the functionality of multiple inheritance.

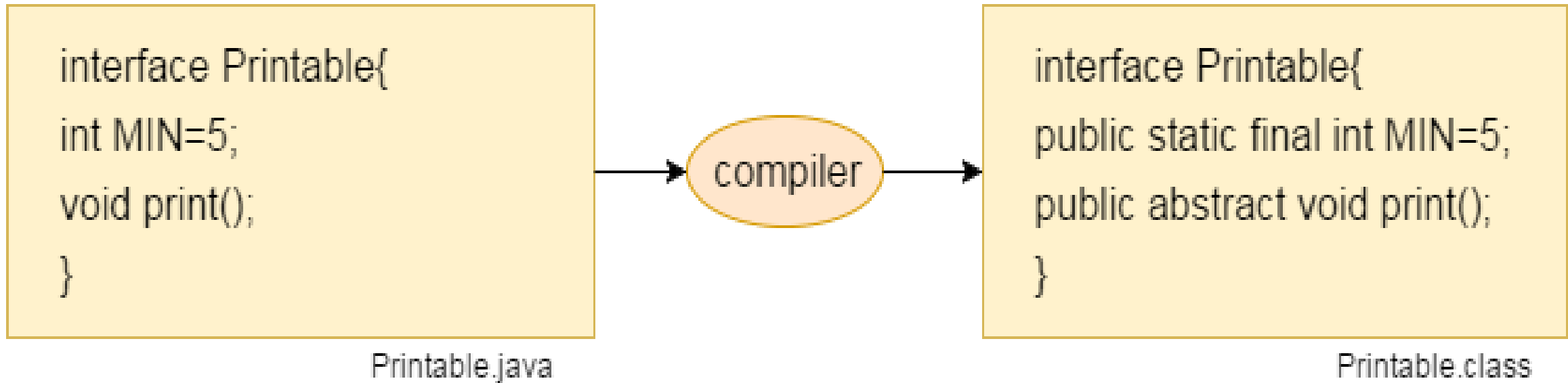
Note:

- Like **abstract classes**, object of interfaces **can not** be created.
- It needs to be implements in a class and its method should be implemented in that class.
- Interface methods are by default abstract and public.

Cont...

- Interface attributes are by default public, static and final.

Example:



Example

```
interface A
{ void print();
}
class B implements A
{ public void print()
```

```
    {  
        System.out.println("Hello");  
    }  
public static void main(String a[])  
{  
    B b1 = new B();  
    b1.print();  
}  
}
```

Cont...

Output:

Hello

Multiple inheritance with the help of Interface

```
interface X
{
    void display();
}
interface Y
{
    void display();
}
class C implements X,
Y
{
    public void display()
    {
        System.out.println("Hello");
    }
    public static void main(String x[])
    {
        C c1 = new C();
        c1.display();
    }
}
```

```
}
```

Output:
Hello

Super Keyword

Super Keyword in Java

- The **super** keyword in Java is a reference variable which is used to refer immediate parent class object.

Usage of Java super Keyword

1. `super` can be used to refer immediate parent class instance variable.
2. `super` can be used to call immediate parent class method.
3. `super()` can be used to call immediate parent class constructor.

1. super is used to refer immediate parent class instance variable.

- We can use super keyword to access the data member or field of parent class.

- It is used if parent class and child class have same fields.

Example

```
class A
{ int a=10;
}
class B extends A
{ int a=20; void
    display()
    {
        System.out.println(a);//prints a of B class
        System.out.println(super.a);//prints a of A class
    }
    public static void main(String x[])
    {
        B b1= new B();
```

```
        b1.display();  
    }  
}
```

Cont...

Output:

20 10

2. super can be used to call parent class method

- The super keyword can also be used to call parent class method.
- It should be used if subclass contains the same method as parent class.
- In other words, it is used if method is overridden.

Example

```
class A
{ void display()
    {
        System.out.println("Class A");
    } }
```

```
class B extends A
{ void display()
    {
        System.out.println("Class B");
        super.display();
    } public static void main(String
X[])
    {
        B b1=new B();
        b1. display();
    }
```

}

Cont...

Output:

Class B

Class A

3. `super()` can be used to call immediate parent class constructor.

- The `super` keyword can also be used to call the parent class constructor.

Example

```
class A
{
    A()
    {
        System.out.println("Class A");
    }
}
class B extends A
{
    B()
    {
        super();
        System.out.println("Class B");
    }
}
```



```
public static void main(String X[])  
{  
    B b1=new B();  
}  
}
```

Cont...

Output:

Class A

Class B

Access Modifiers in Java

Modifiers in Java

- There are two types of modifiers in Java:
- **Access modifiers** (Visibility Control).
- **Non-access modifiers**.

1. Access modifiers

- The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class.
- There are four types of Java access modifiers:
- **Private:** The access level of a private modifier is only within the class.

It cannot be accessed from outside the class.

Cont...

- **Default:** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.

- **Protected:** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.

Cont...

- **Public:** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Cont...

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N

Public	Y	Y	Y	Y
---------------	---	---	---	---

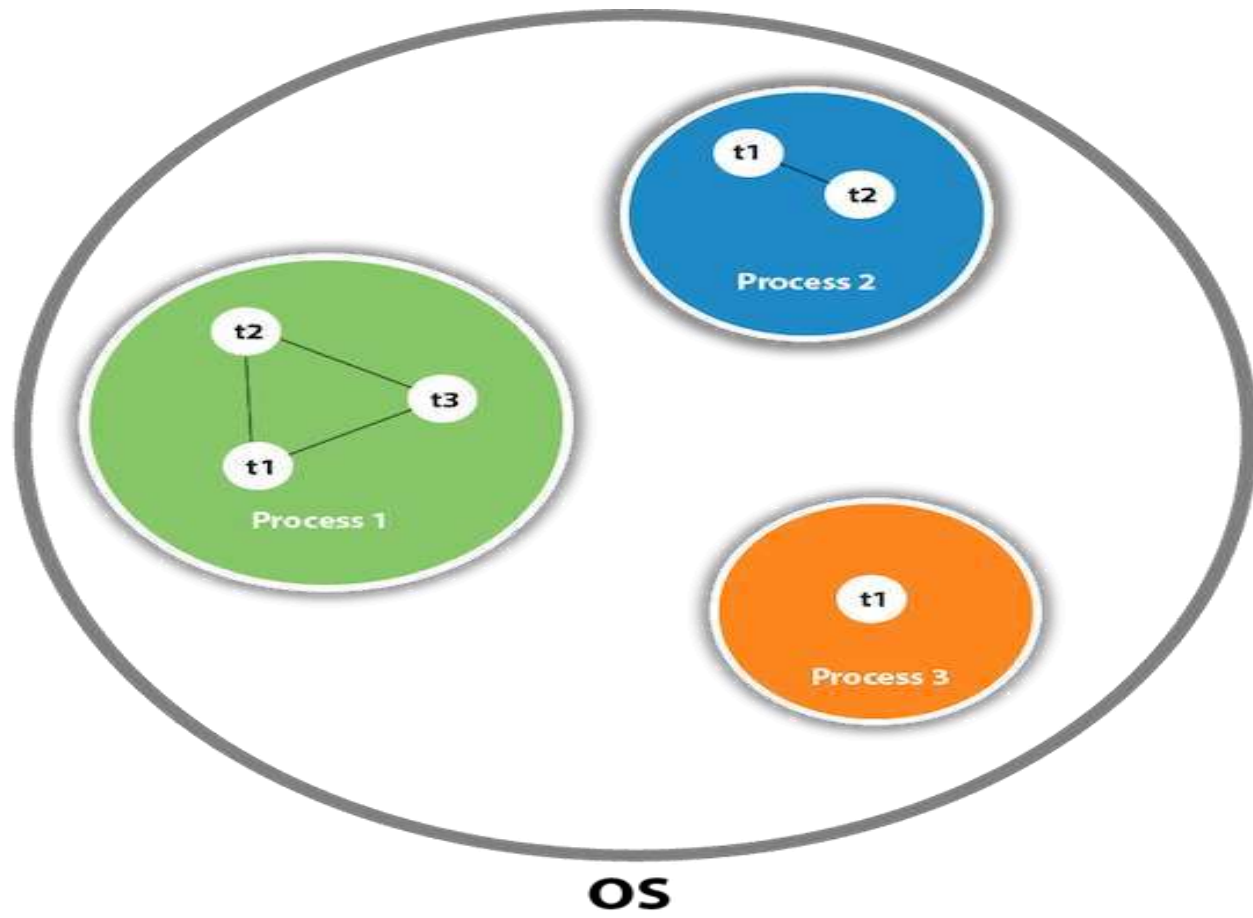
2. Non-access modifiers

- There are many non-access modifiers, such as static, abstract, synchronized, native, volatile, transient, etc.

Multithreading in Java

Thread

- A thread is a lightweight sub-process.



Cont...

- The smallest unit of processing.
- Threads are independent

Cont...

- If there occurs exception in one thread, it doesn't affect other threads.
- It uses a shared memory area.

Multithreading

- Multithreading in Java is a process of executing multiple threads simultaneously.

Advantages of Java Multithreading

- You can perform many operations together, so it saves time.

Cont...

- It doesn't block the user because threads are independent and you can perform multiple operations at the same time.

- Threads are independent, so it doesn't affect other threads if an exception occurs in a single thread.

Application of Multi Threading

- Games
- Animations

Life cycle of a Thread (Thread States)

- There are 5 states in the life cycle of thread .
 1. New
 2. Runnable
 3. Running
 4. Non-Runnable (Blocked)
 5. Terminated

**Note: The life cycle of the thread in java
is controlled by JVM.**

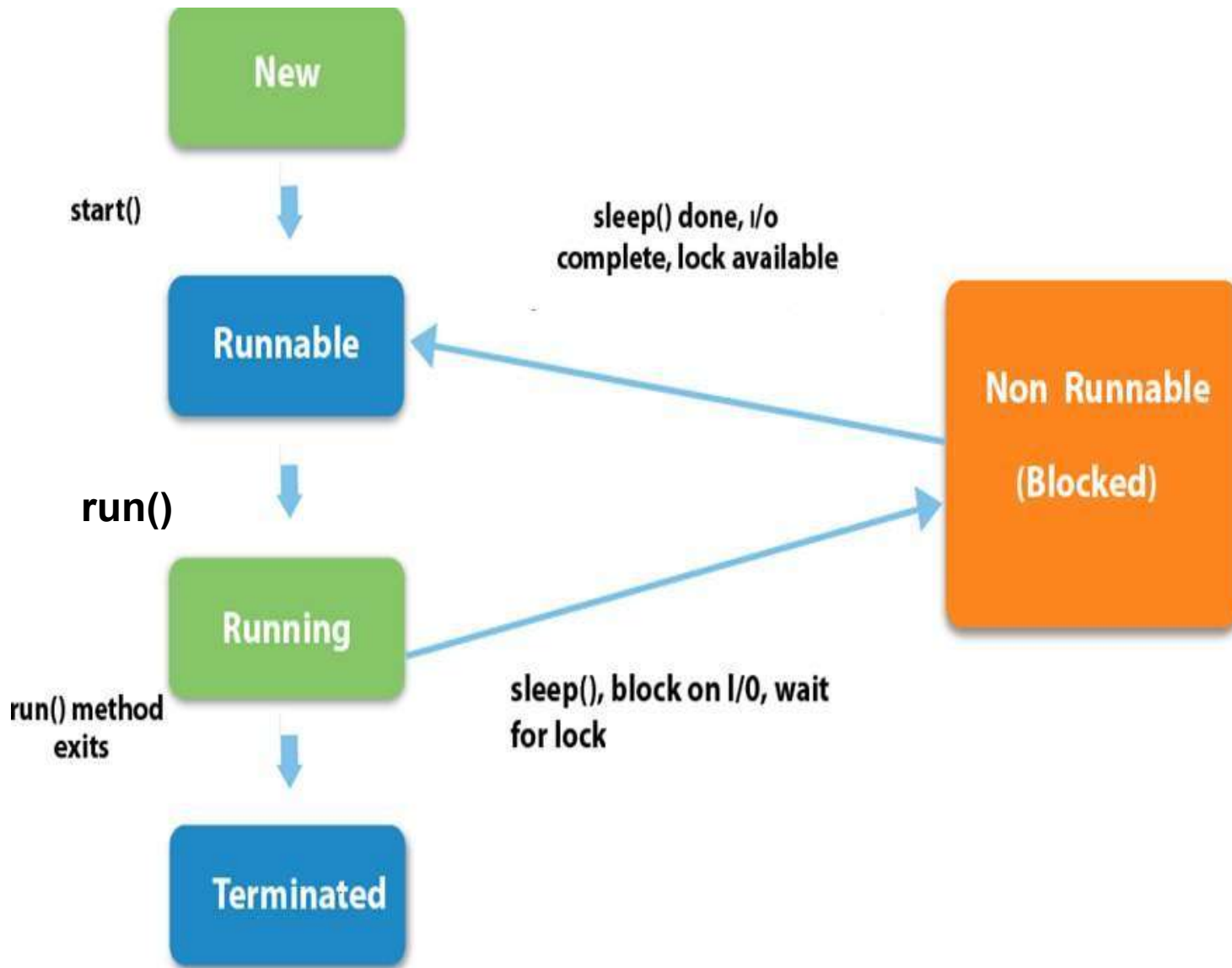


Diagram for Life Cycle of Thread

Cont...

1. New:

The thread is in new state if you create an instance **(object)** of Thread class but before the calling of **start()** method.

Cont...

2. Runnable:

The thread is in runnable state after calling of **start()** method, but the thread scheduler has not selected it to be the running thread.

Cont...

3. Running:

The thread is in running state if the thread scheduler has selected it and in this stage **run()** method is called.

Cont...

4. Non-Runnable (Blocked):

This is the state when the thread is still alive, but is currently not eligible to run.

5. Terminated:

A thread is in terminated or dead state when its **run()** method exits.

How to create thread

- There are two ways to create a thread:

1. By extending Thread class
2. By implementing Runnable interface.

1. By extending Thread class

Thread class:

- Thread class provide **constructors** and **methods** to create and perform operations on a thread.
- Thread class extends **Object class** and implements **Runnable interface**.

Constructors of Thread class

- Thread()
- Thread(String name)

Methods of Thread class

1. **public void start():** starts the execution of the thread. JVM calls the run() method of the thread.
2. **public void run():** is used to perform action for a thread. It is declared in Runnable Interface

Cont...

3. **public static void sleep(long millis):** It sleeps a thread for the specified amount of time.
4. **public void final setPriority(int newPriority):** It set the priority of the thread.

5. **public int getPriority():** It returns the priority of the thread.

Example

```
class A extends Thread
{ public void run()
    {
        System.out.println("Thread is running");
    }

    public static void main(String a[])
    {
        A t1 = new A();
        t1.start();
    }
}
```

```
}
```

Output: Thread is running

2. By implementing Runnable interface.

Runnable interface:

- It contains only run() Method.

Example

```
class A implements Runnable
{ public void run()
  {
    System.out.println("Runnable Interface");
  } public static void main(String
    a[])
  {
    A t1 = new A();
    Thread t2 = new Thread(t1);
    t2.start();
  }
}
```

Output:

Runnable Interface

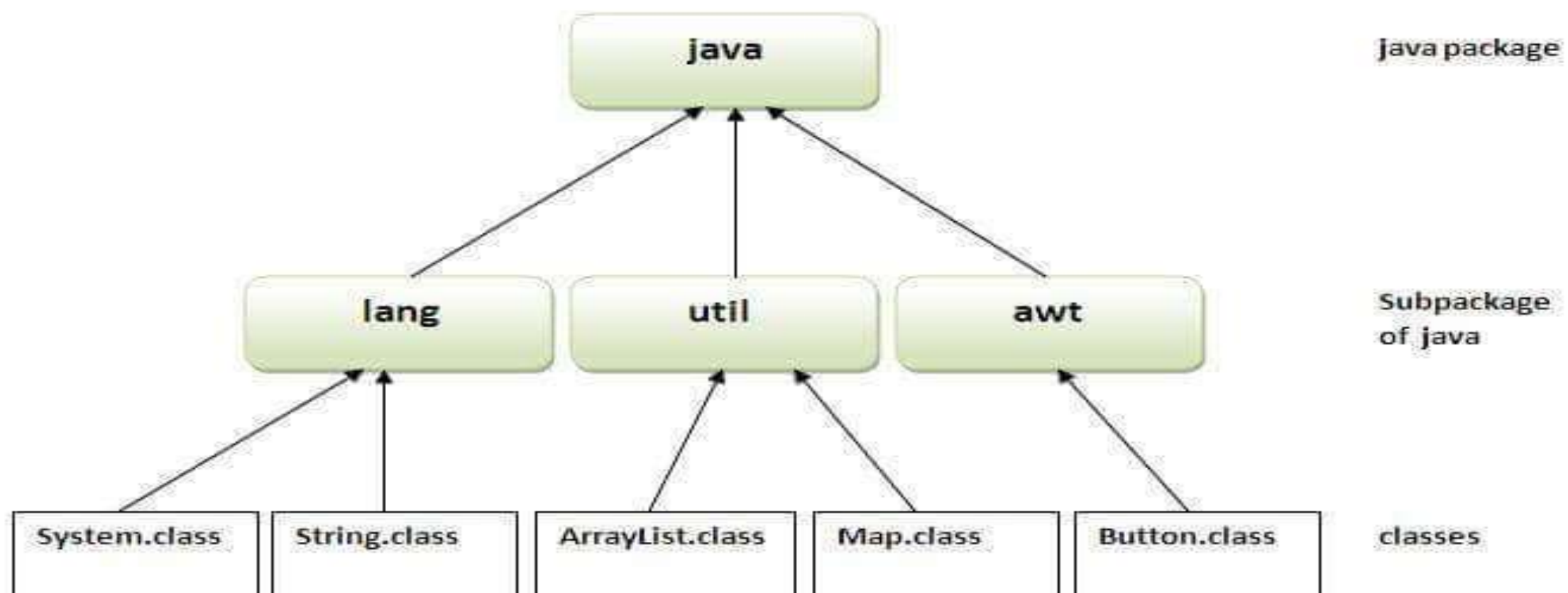
Java Package

Java Package

- A **java package** is a group of similar types of classes, interfaces and sub-packages.
- Package in java can be categorized in two form, built-in (pre-defined) package and user-defined package.

- There are many built-in (pre-defined) packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Cont...



Advantage of Java Package

- Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- Java package provides access protection.
- Java package removes naming collision.

Example of java package (User-defined package)

- The **package keyword** is used to create a package in java.

//Simple.java

```
package mypack;  
public class Simple  
{ public static void main(String a[])  
    {  
        System.out.println("Welcome to package");  
    }  
}
```

}

How to compile java package

- **Syntax**

`javac -d directory javafilename.java`

- **For example**

`javac -d . Simple.java`

- The -d is a switch that tells the compiler where to put the .class file i.e. it represents destination. You can use any directory name like

d:/abc etc. If you want to keep the package within the same directory, you can use . (dot).

How to run java package program

- For example
- java mypack.Simple

Applet

Applet

- An **applet** is a Java program that can be executed by java-enabled web-browser (if a browser is supporting the applets) or applet can be executed by using the **appletviewer** utility provided with JDK .

- Applet is a java program that is embedded in the webpage (HTML page) to generate the dynamic content.

Advantage of Applet

- It works at client side so less response time.

- It can be executed by browsers running under many plate-forms, including Linux, Windows, Mac Os etc.

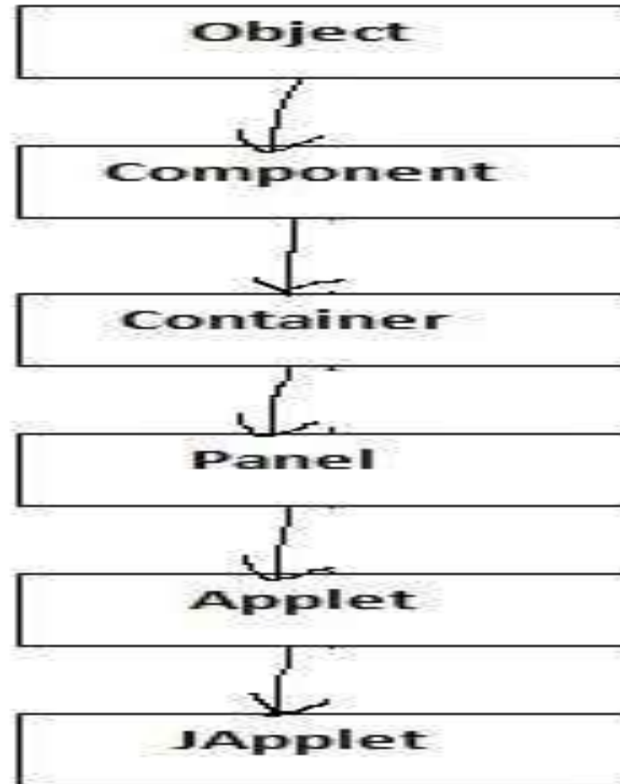
Drawback of Applet

- Plugin is required at client browser to execute applet.

Differences between an applet and a standalone Java application

- An applet is a Java class that extends theApplet class.
- Applets are designed to be embedded within an HTML page.
- There is no main() methods in an Applet class.

Hierarchy of Applet



Applet class

- java.applet package containApplet class.

- **Method of Applet class**

1. **public void init() :** This is the first method called during the lifecycle of an applet.

- In this method, we initialize variables that are going to be needed throughout the life of an applet.

Cont...

2. public void start():

- It is called immediately after the init() method is called.
- It is used to start the Applet.
- It is also called whenever the applet is maximized or moving from one tab to another tab in the browser.

Cont...

3. **public void stop():**

- The stop() method, stops the execution of the applet.
- The stop () method is called whenever the applet is minimized or moving from one tab to another in the browser.

Cont...

4. public void destroy():

- The destroy() method destroys the applet after its work is done.
- It is called when the applet window is closed or when the tab containing the webpage is closed.

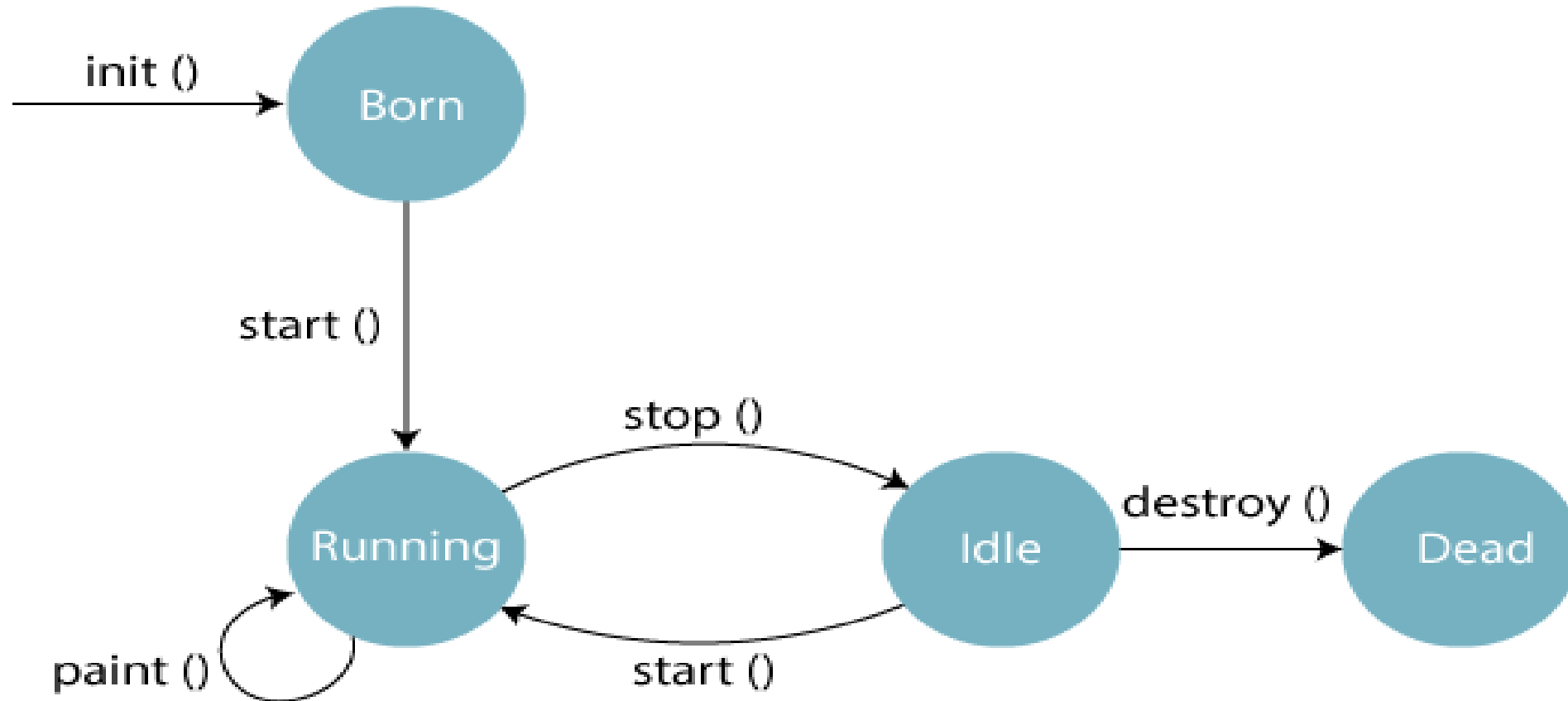
Component class

- java.awt package contain Component class.
- The Component class provides 1 life cycle method of applet.
- **public void paint(Graphics g):** This method is used to paint the Applet.

Lifecycle of Java Applet

1. Applet is initialized.
2. Applet is started.
3. Applet is painted.
4. Applet is stopped.
5. Applet is destroyed.

Cont...



Cont...

- There are five methods of an applet life cycle, and they are:

1. `init()`
2. `start()`
3. `paint()`
4. `stop()`
5. `destroy()`

1. `init()`:

Cont...

- This method is the first method to run.
- This method will be called only one time.
- This method initializes the applet.
- When this method is called then applet will enter in born state.

Cont...

2. **start():**

- When this method is called then applet will enter in running state.
- It is called immediately after the `init()` method is called.
- It is also called whenever the applet is maximized or moving from one tab to another tab in the browser.

3. **stop():**

Cont...

- When this method is called then applet will enter in idle state.
- The stop() method, stops the execution of the applet.
- The stop () method is called whenever the applet is minimized or moving from one tab to another in the browser.

Cont...

4. **paint():**

- It is used to draw shapes like circle, square etc., in the applet.
- It is executed after the start() method.

5. **destroy():**

- When this method is called then applet will enter in dead state.

Cont...

- The `destroy()` method destroys the applet after its work is done.
- It is called when the applet window is closed or when the tab containing the webpage is closed.
- It removes the applet object from memory and is executed only once.
- We cannot start the applet once it is destroyed.

Example

```
//A.java
import java.applet.Applet;
import java.awt.Graphics;
public class A extends Applet
{ public void paint(Graphics
  g)
  {
    g.drawString("Hello",50,50);
  }
}
/* <applet code= "A.class" width=300 height=300>
```

Cont...

</applet>

*/

Cont...

- To execute the applet by appletviewer tool, write in command prompt:
- D:\> javac A.java
- D:\> appletviewer A.java